

WOXSEN UNIVERSITY
SCHOOL OF TECHNOLOGY



INTELLIGENT MAZE SOLVER USING AI SEARCH ALGORITHMS

"A Comparative Study of BFS, DFS and A* Search for Maze Navigation"

PROJECT-BASED LEARNING REPORT

Foundations of Artificial Intelligence

Course code: 25TU03MJM2 | Semester: III | Academic year: 2026-2027

Submitted by

Sindhu T. = 25WU0102316

Reddy Keerthana = 25WU0101107

Uday Soni = 25WU0101148

Under the guidance of: Dr. Steven Raj

October 2026

DECLARATION

We declare that the project report titled "Intelligent Maze Solver Using AI Search Algorithms" represents our group's work for the Foundations of Artificial Intelligence course. The report describes the maze-solving agent presented in our project slides. We have distinguished observed results from design aims and cited the published sources used for the conceptual background.

Each group member is responsible for reviewing the final submission and confirming that it accurately reflects the implemented project. The faculty guide's name, signatures, and institutional checks should be completed before submission.

Place: Hyderabad

Date: 5/10/2026

Signatures of the candidates:

Sindhu T. = 25WU0102316

Reddy Keerthana = 25WU0101107

Uday Soni = 25WU0101148

CERTIFICATE

This is to certify that the report titled "Intelligent Maze Solver Using AI Search Algorithms" is submitted by the students listed below as part of the course Foundations of Artificial Intelligence at Woxsen University during the academic year 2026-2027.

Sindhu T. - 25WU0102316

Reddy Keerthana - 25WU0101107

Uday Soni - 25WU0101148

The work has been carried out under my guidance, subject to verification and approval by the faculty member concerned.

Faculty guide name and designation: Dr. Steven Raj

Faculty signature: _____ Date: _____

ACKNOWLEDGEMENT

We would like to thank our Foundations of Artificial Intelligence faculty for helping us connect ideas from class to a working program. Building the maze solver made us think beyond "find a way out." We had to decide how a maze should be represented, which search strategy to use, and how to compare strategies fairly.

We also thank our classmates for their feedback on our presentation. Each member of the group contributed to the planning, implementation, or explanation of the project. This report brings those parts together in one place.

ABSTRACT

Solving a maze by hand works for small puzzles but becomes slow and error-prone as the maze grows. An intelligent agent should instead search the space systematically. This project models a maze as a formal search problem, with states, actions, a transition model, a goal test and a path cost, and solves it with three classic search algorithms: Breadth-First Search (BFS), Depth-First Search (DFS) and A*.

The maze is a grid of walls and open cells. The agent moves up, down, left or right, and each move costs one step. All three algorithms run through the same pipeline (build the grid model, run the search, track explored cells, reconstruct the path, visualize the result), and only the search strategy changes. This keeps the comparison fair.

On our sample test maze, BFS explored 42 cells and found a path of 12 steps. DFS explored 35 cells and found a path of 19 steps. A* explored 18 cells and found a path of 12 steps. A* therefore found the shortest path while exploring fewer than half the cells BFS visited. DFS explored fewer cells than BFS but returned a longer route. These figures come from a single test maze, so they illustrate the behavior of each algorithm rather than prove a general ranking.

The project shows how problem formulation, uninformed search, informed search and heuristics fit together, and why heuristics matter in navigation systems.

TABLE OF CONTENTS

CERTIFICATE	3
ACKNOWLEDGEMENT	4
ABSTRACT	5
TABLE OF CONTENTS	6
LIST OF FIGURES	7
LIST OF TABLES	8
1 INTRODUCTION	9
1.1 Background	9
1.2 Motivation and relevance	9
1.3 Scope of the report	9
1.4 Why the three algorithms matter	10
1.5 What we learned from the project	10
2 PROBLEM STATEMENT AND OBJECTIVES	11
2.1 Problem definition	11
2.2 Objectives	11
2.3 Constraints and assumptions	11
2.4 Success criteria	11
2.5 Boundaries of the problem	12
3 LITERATURE REVIEW AND BACKGROUND	13

3.1 Intelligent agents and search problems	13
3.2 Uninformed search: BFS and DFS	13
3.3 Informed search: A*	13
3.4 Comparison of approaches	14
3.5 Research gap for this classroom project	14
4 METHODOLOGY AND SYSTEM DESIGN	15
4.1 Overall architecture	15
4.2 Problem formulation	15
4.3 Breadth-First Search procedure	16
4.4 Depth-First Search procedure	16
4.5 A* search procedure	16
4.6 Path reconstruction and tracking	17
4.7 Handling edge cases	17
4.8 Evaluation design	17
5 IMPLEMENTATION	18
5.1 Modules and interface	18
5.2 Maze representation	18
5.3 BFS implementation	18
5.4 DFS implementation	18
5.5 A* implementation	18

5.6 Pseudocode (A*)	19
5.7 Visualization	19
6 RESULTS AND DISCUSSION	20
6.1 Observed results on the test maze	20
6.2 Discussion	20
6.3 What the comparison can and cannot show	21
6.4 Limitations	21
6.5 Proposed test protocol	21
7 CONCLUSION AND FUTURE SCOPE	22
7.1 Conclusion	22
7.2 Real-world applications	22
7.3 Future scope	22
REFERENCES	23
APPENDIX A: PROPOSED EXTENDED EVALUATION PLAN	24
APPENDIX B: DESIGN DECISIONS AND OPEN QUESTIONS	25

LIST OF FIGURES

Figure 1. Maze grid with start, goal, walls and solved path

Figure 2. BFS exploration

Figure 3. DFS exploration

Figure 4. A* exploration

Figure 5. Nodes explored comparison chart

Figure 6. Path length comparison chart

LIST OF TABLES

Table 1. Comparison of BFS, DFS and A*

Table 2. System pipeline stages

Table 3. Search problem formulation

Table 4. Evaluation measures

Table 5. Results on the test maze

Table 6. Proposed test scenarios

1 INTRODUCTION

1.1 Background

A maze is a classic way to describe navigation problems. It has walls, a start point, a goal, and open cells through which a path must be found. The same idea appears in route planning, robot navigation, video games and network routing.

An intelligent agent connects observations to actions in pursuit of a goal. For our project the goal is to travel from the start cell to the goal cell through open cells. This does not require a camera, a physical robot or machine learning. It requires a representation of the maze, a way to generate legal moves, and a search strategy that decides which cell to examine next [1], [2].

We model the maze as a grid because the state is visible and each step of the search can be followed cell by cell. The agent can move up, down, left or right into an adjacent open cell. Diagonal movement is excluded.

1.2 Motivation and relevance

Solving large or complex mazes manually is slow and error-prone. Guessing and backtracking wastes effort and gives no guarantee about the quality of the route. A systematic search can guarantee that a path is found if one exists, and some algorithms can also guarantee that it is the shortest.

Different search algorithms behave very differently on the same maze. One may find the shortest path but visit many cells. Another may use little memory but return a long, winding route. A third may use a heuristic to reach the goal while exploring far fewer cells. Seeing these trade-offs side by side is the main educational value of the project.

The project is a simulation on a grid, so it does not show how a commercial navigation system or a physical robot performs. Its relevance is that the same ideas of state, action, path cost and heuristic appear in GPS routing, warehouse robots and game characters.

1.3 Scope of the report

The report first defines the problem and objectives. It then relates the design to intelligent agents, state-space search, uninformed search and heuristic search. Later sections explain the methodology, the implementation of the three algorithms, and the results of the comparison on a sample maze.

The scope is a discrete, two-dimensional grid maze with uniform step cost, known walls, and a single start and goal. Weighted terrain, moving obstacles and three-dimensional mazes are outside the present scope and are listed as future work.

1.4 Why the three algorithms matter

BFS, DFS and A* were chosen because together they cover the two main families of search. BFS and DFS are uninformed: they use only the structure of the maze and no knowledge about where the goal is. A* is informed: it uses a heuristic estimate of the distance to the goal.

The three algorithms are not a complete benchmark of all search methods. They illustrate how the amount of knowledge the agent uses changes what it explores. Performance claims in this report are limited to the test maze we ran, and Appendix A describes how a broader comparison could be done.

1.5 What we learned from the project

At first glance maze solving looks like "keep walking until you reach the exit." During the design it became clear that the answer depends on how the problem is formulated. Once the maze is expressed as states, actions and a goal test, the same program structure can run any search strategy. Changing only the strategy is what makes a fair comparison possible.

We also learned that "finds a path" and "finds the best path" are different claims, and that exploring fewer cells does not have to cost path quality when a good heuristic is used.

2 PROBLEM STATEMENT AND OBJECTIVES

2.1 Problem definition

The maze contains walls, open cells, a start cell and a goal cell. The agent must find a sequence of moves from the start to the goal that never enters a wall. The agent should do this automatically, and its result should be easy to inspect.

The difficulty is that there are many possible routes, and an algorithm must decide which cells to examine and in what order. Our central question is: how do uninformed search (BFS, DFS) and informed search (A*) compare in the number of cells explored and the length of the path found when solving the same maze?

2.2 Objectives

1. Model the problem: represent any maze as a formal search problem with states, actions, transitions and a goal test.
2. Implement algorithms: build and run uninformed (BFS, DFS) and informed (A*) search over the maze.
3. Compare performance: evaluate path length, nodes explored and time taken across the algorithms.
4. Visualize the solution: display the maze, the explored region and the final path clearly.

2.3 Constraints and assumptions

The grid is discrete and movement is restricted to four directions. Every move costs one step, and walls cannot be crossed. The maze is fully known to the agent before the search starts, so no unknown-cell or exploration policy is assumed. The start and goal cells are fixed for a run.

2.4 Success criteria

At the functional level, each algorithm should return a valid path from start to goal that avoids walls, or report that no path exists. The explored cells and the final path should be displayed.

At the comparative level, the algorithms must be run on the same maze with the same start and goal, and the same measures must be recorded for each: nodes explored, path length and time taken. BFS and A* should return a shortest path. If they do not, that points to an implementation error.

2.5 Boundaries of the problem

This project does not solve every navigation problem. The maze is a small grid with known walls. We have no sensors, no mapping from images and no motor control, and we do not model turning cost, speed or surface type. The results show how the algorithms behave on the grid model, not on a physical maze.

3. LITERATURE REVIEW AND BACKGROUND

3.1 Intelligent agents and search problems

Russell and Norvig describe an intelligent agent in terms of what it perceives, what actions it can take, its goal and its environment [1]. A maze-solving agent perceives its cell and its open neighbors, chooses a move, and stops at the goal. Poole and Mackworth show how such problems can be written as states, actions and a goal, so that a general search procedure can solve them [2].

A search problem is defined by its initial state, actions, transition model, goal test and path cost [1]. In our maze these are the start cell, the four moves, the resulting cell (if not blocked), the test "current cell equals goal cell" and a cost of one per step.

3.2 Uninformed search: BFS and DFS

Breadth-First Search expands the shallowest unexpanded cell first, using a FIFO queue. In an unweighted maze it is complete and returns a shortest path, but it must hold the whole frontier in memory, which grows quickly in open mazes [1], [3]. On a grid its running time is proportional to the number of cells and moves checked, $O(V + E)$.

Depth-First Search follows one branch as deep as possible before backtracking, using a stack or recursion. It needs less memory than BFS, since it stores only the current branch and its siblings, but it is not guaranteed to find the shortest path [1], [3]. The first path it finds depends on the order in which moves are tried.

Moore and Lee studied shortest-path routing through mazes and grids in 1959 and 1961, and their wave-propagation approach is essentially breadth-first search on a grid [4], [5].

3.3 Informed search: A*

Hart, Nilsson and Raphael introduced A* as a heuristic method for finding minimum-cost paths [6]. A* evaluates a node with

$$f(n) = g(n) + h(n)$$

where $g(n)$ is the cost from the start to n and $h(n)$ is an estimate of the remaining cost to the goal. In a four-direction grid with uniform step cost, the Manhattan distance $|x_{\text{goal}} - x| + |y_{\text{goal}} - y|$ is a

natural heuristic. It never overestimates the true remaining cost, so it is admissible, and it is also consistent. With an admissible heuristic, A* returns an optimal path [1], [6].

Dijkstra's algorithm also finds shortest paths, but it expands cells in order of cost so far only and uses no estimate of the goal direction [7]. A* can be seen as Dijkstra's algorithm guided by a heuristic. Walls must still be checked when neighbors are generated, because the heuristic alone does not detect them. A wall can force a detour, so the Manhattan value is an estimate and not the final route length.

3.4 Comparison of approaches

Table1:Comparison of BFS, DFS and A*

1

Property	BFS	DFS	A*
Type	Uninformed	Uninformed	Informed
Data structure	FIFO queue	Stack or recursion	Priority queue (open list)
Shortest path (unit cost)	Guaranteed	Not guaranteed	Guaranteed (admissible heuristic)
Memory use	High	Low	Moderate to high
Uses knowledge of the goal	No	No	Yes, through the heuristic
Typical weakness	Explores many cells	Long, winding paths	Depends on heuristic quality

3.5 Research gap for this classroom project

The sources explain agents, search, shortest paths and heuristics individually. Our assignment puts BFS, DFS and A* in one program that runs on the same maze, so a viewer can compare their explored cells and paths directly.

There is no claim that this is a new algorithm. The project uses established methods on a deliberately small problem. It is worthwhile for learning because the difference between exploring everything, going deep, and going toward the goal becomes visible. A broader evaluation would test many mazes of different sizes and wall densities, and would add other heuristics. We did not

conduct that wider experiment, so the literature is used here to explain the design and not to claim general superiority.

4 METHODOLOGY AND SYSTEM DESIGN

4.1 Overall architecture

Table 2. System pipeline stages

Step	Stage	Responsibility
1	Input maze	Read the grid with walls, open cells, start and goal
2	Build grid model	Convert the maze into cells and legal neighbors
3	Run search algorithm	Apply BFS, DFS or A* (the only step that changes)
4	Track explored cells	Record every cell expanded, in order
5	Reconstruct path	Follow parent links from goal back to start
6	Visualize result	Draw the maze, explored region and final path

Because only the search strategy changes between runs, any difference in the results comes from the algorithm and not from the maze, the start and goal, or the display. This keeps the comparison fair.

4.2 Problem formulation

Table 3. Search problem formulation

Element	Definition in the maze
State	A single cell (row, column) where the agent is located
Initial state	The agent's starting cell
Actions	Move Up, Down, Left or Right to an adjacent open cell
Transition model	The new cell, if inside the grid and not a wall
Goal test	True when the current cell equals the goal cell

Path cost	Number of steps; each move costs 1
-----------	------------------------------------

The environment is discrete, deterministic (the same move in the same state always gives the same result), fully observable (the whole maze is known) and sequential (each move changes the later choices).

4.3 Breadth-First Search procedure

1. Put the start cell in a FIFO queue and mark it visited.
2. Remove the cell at the front of the queue.
3. If it is the goal, stop and reconstruct the path.
4. Otherwise add every unvisited open neighbor to the back of the queue, record each neighbor's parent, and mark it visited.
5. Repeat until the goal is found or the queue is empty (no path).

Since cells are expanded level by level, the first time the goal is reached is by a path with the fewest steps.

4.4 Depth-First Search procedure

1. Push the start cell on a stack (or use recursion) and mark it visited.
2. Pop a cell. If it is the goal, stop and reconstruct the path.
3. Push its unvisited open neighbors in a fixed order and record their parents.
4. Continue along the newest branch. When a dead end is reached, backtrack to the last cell with an unexplored neighbor.
5. Stop when the goal is found or the stack is empty.

The route returned depends on the move order (for example Up, Right, Down, Left), and it need not be the shortest.

4.5 A* search procedure

1. Keep an open list of frontier cells ordered by $f(n) = g(n) + h(n)$, and a closed set of expanded cells.
2. Always expand the open cell with the lowest $f(n)$.

3. For each open neighbor, compute the new g value. If it is lower than any value found before, update the neighbor's g , h , f and parent.
4. Stop when the goal cell is expanded, then reconstruct the path using the parent links.

Here $g(n)$ counts steps already taken and $h(n)$ is the Manhattan distance to the goal. Since the heuristic never overestimates the cost in this grid, the returned path is the shortest.

4.6 Path reconstruction and tracking

Each cell stores a parent. When the goal is reached, the program follows parents back to the start and reverses the list to obtain the path. During the search, every expanded cell is added to an explored list. Its length is the "nodes explored" measure, and the list is also used to draw the explored region.

4.7 Handling edge cases

No path exists: if the frontier becomes empty without reaching the goal, the program should report that the maze cannot be solved instead of returning a partial path.

Start equals goal: the path has length zero and the search stops immediately.

Start or goal inside a wall: the input should be rejected.

Ties in A*: two cells may have equal f values, so a fixed tie-break rule (for example smaller h first) makes runs repeatable.

Move order in DFS: the order of neighbors should be fixed and recorded, since it changes the path.

4.8 Evaluation design

Table 4. Evaluation measures

Measure	Definition	Reason to collect it
Nodes explored	Number of cells expanded before stopping	Shows search effort and memory pressure
Path length	Number of steps in the returned path	Shows the quality of the solution
Time taken	Run time of the search	Shows the practical cost
Optimality	Whether path length equals the known shortest length	Correctness check

5. IMPLEMENTATION

5.1 Modules and interface

The project is implemented in [language, for example Python or JavaScript]. It consists of [list your files or functions, for example maze representation, BFS, DFS, A*, visualizer]. The interface displays the maze grid with colors for walls, open cells, explored cells and the solved path.

5.2 Maze representation

The maze is stored as a two-dimensional grid of size [rows x columns] in which each cell is either a wall or an open cell. The program knows the start cell and the goal cell. A helper function returns the legal neighbors of a cell: the four adjacent cells that lie inside the grid and are not walls. All three algorithms use this same function, which is why their results are comparable.

5.3 BFS implementation

BFS uses a queue [for example collections.deque] and a visited set. Each neighbor is stored with its parent so that the path can be rebuilt after the goal is found.

5.4 DFS implementation

DFS uses a stack [or recursion] and a visited set. The order in which neighbors are pushed is fixed [state your order, for example Up, Right, Down, Left].

5.5 A* implementation

A* uses a priority queue [for example heapq] ordered by $f = g + h$, with the Manhattan distance as the heuristic. The implementation keeps the best known g for each cell and updates a cell when a cheaper route is found.

5.6 Pseudocode (A*)

open = priority queue containing start, with $f = h(\text{start})$

$g[\text{start}] = 0$

```

while open is not empty:

    n = cell in open with lowest f

    if n is the goal:

        return reconstruct_path(n)

    add n to closed

    for each neighbor m of n that is not a wall:

        new_g = g[n] + 1

        if m not in g or new_g < g[m]:

            g[m] = new_g

            f[m] = new_g + manhattan(m, goal)

            parent[m] = n

            add m to open

return "no path"

```

BFS and DFS follow the same structure, using a FIFO queue and a stack respectively instead of the priority queue, and with no heuristic.

5.7 Visualization

The visualization shows the maze, the explored cells and the final path. Different colors for walls, open cells, explored cells and the solved path make it easy to see how far each algorithm searched before finding the goal.

[Insert screenshots here: Figure 1 maze with start and goal, Figure 2 BFS result, Figure 3 DFS result, Figure 4 A* result.]

6. RESULTS AND DISCUSSION

6.1 Observed results on the test maze

Table 5. Results on the test maze

Algorithm	Nodes explored	Path length
BFS	42	12
DFS	35	19
A*	18	12

These are results from one test maze. They are not an average over many mazes.

6.2 Discussion

BFS found a path of 12 steps. This is the shortest path in this maze, as expected for an unweighted grid, but it explored 42 cells because it spreads out in every direction until it reaches the goal.

DFS explored 35 cells, fewer than BFS, but its path was 19 steps, about 58 percent longer than the shortest. This matches the theory: DFS commits to one branch and may find a long route first. In another maze, or with a different move order, DFS could do better or worse.

A* explored 18 cells, which is about 43 percent of what BFS explored (18 out of 42), and still found the 12-step shortest path. The Manhattan heuristic guided the search toward the goal, so cells far from the goal were never expanded.

6.3 What the comparison can and cannot show

The results agree with the theory: BFS and A* returned the shortest path, DFS did not, and A* needed the fewest expansions. They do not show that A* always explores less than half of what BFS does. The saving depends on the maze layout and on how well the heuristic fits it. In a maze with long dead-end corridors that point toward the goal, the Manhattan heuristic can be misleading and A* may explore many more cells.

Nodes explored is also only a measure of effort. A* does more work per expanded cell (it maintains a priority queue and computes h), so fewer expansions do not automatically mean less running time on a small grid. Time should be measured separately. [Add your measured times if you have them.]

6.4 Limitations

- The experiment uses a single small maze.
- The maze is fully known and static, with no hidden or moving walls.
- Costs are uniform, with no weighted terrain.
- Running time was not measured (or: running time was measured once and not averaged over repeated runs).
- The DFS result depends on the chosen move order.

6.5 Proposed test protocol

To make the comparison stronger, save several mazes (open, dense, with dead ends, with no solution, with a long winding solution), run all three algorithms on each with identical start and goal, and report the measures per maze before averaging. See Appendix A.

7. CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

Our Intelligent Maze Solver turns maze navigation into a search problem and solves it with BFS, DFS and A*. Formulating the maze with states, actions, a goal test and a unit step cost allowed one pipeline to run all three strategies, so the comparison depends only on the algorithm.

On our test maze BFS and A* found the shortest path of 12 steps, DFS found a longer path of 19 steps, and A* explored only 18 cells against 42 for BFS and 35 for DFS. This shows the trade-offs between completeness, optimality, memory and efficiency, and the power of a good heuristic. These principles underlie navigation in GPS, robotics, games and network routing. The numbers come from a single maze, so a general claim would need tests on many mazes.

7.2 Real-world applications

- GPS and vehicle route navigation
- Warehouse and delivery robot path planning
- Non-player character movement in games
- Network packet routing

7.3 Future scope

- Add weighted terrain, where some cells cost more to cross.
- Support dynamic, moving obstacles and replanning.
- Compare additional heuristics for A*, for example Euclidean distance.
- Add other algorithms such as Dijkstra, Greedy Best-First and Iterative Deepening.
- Extend to unknown mazes that the agent discovers while moving.
- Extend to 3D maze environments.

REFERENCES

- [1] S. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, 4th ed. Pearson, 2021.
- [2] D. L. Poole and A. K. Mackworth, Artificial Intelligence: Foundations of Computational Agents, 3rd ed. Cambridge University Press, 2023. <https://artint.info/3e/html/ArtInt3e.html>
- [3] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, 4th ed. MIT Press, 2022.
- [4] E. F. Moore, "The shortest path through a maze," in Proc. Int. Symp. on the Theory of Switching, Harvard University Press, 1959, pp. 285-292.
- [5] C. Y. Lee, "An algorithm for path connections and its applications," IRE Trans. Electronic Computers, vol. EC-10, no. 3, pp. 346-365, 1961.
- [6] P. E. Hart, N. J. Nilsson and B. Raphael, "A Formal Basis for the Heuristic Determination of Minimum Cost Paths," IEEE Trans. Systems Science and Cybernetics, vol. 4, no. 2, pp. 100-107, 1968. doi: 10.1109/TSSC.1968.300136.
- [7] E. W. Dijkstra, "A note on two problems in connexion with graphs," Numerische Mathematik, vol. 1, pp. 269-271, 1959. doi: 10.1007/BF01386390.

APPENDIX A: PROPOSED EXTENDED EVALUATION PLAN

Table 6. Proposed test scenarios

Scenario	Maze feature	Question
A	Open maze with few walls	Does A* still explore fewer cells than BFS?
B	Dense maze with a long winding solution	How does the DFS path length compare with the shortest path?
C	Many dead ends in the direction of the goal	Can the Manhattan heuristic mislead A*?
D	No solution (goal enclosed by walls)	Do all algorithms correctly report no path?
E	Larger maze (for example 50 x 50)	How do nodes explored and time grow with size?
F	Start equals goal	Is the zero-length case handled?

This appendix describes proposed tests, not tests already completed. Each maze should be saved with its grid size, wall positions, start and goal, so that the same state can be given to all three algorithms.

For each scenario record nodes explored, path length, time over repeated runs, and whether the path is optimal. Report each maze before calculating averages, since a single average can hide a case where an algorithm does badly.

APPENDIX B: DESIGN DECISIONS AND OPEN QUESTIONS

Which heuristic is used? We used the Manhattan distance, since moves are four-directional with unit cost. With diagonal moves or weighted cells, this heuristic would have to be reconsidered.

How are ties handled? Equal f values in A*, and neighbor order in DFS, should follow a fixed documented rule so that runs can be repeated.

What counts as an explored node? We count a cell when it is expanded (removed from the queue, stack or open list), and the same rule should be used for all three algorithms.

What does the model leave out? The simulation uses symbolic information about cells. A real robot would have to estimate its position and detect walls with noisy sensors. Our report does not claim that the program performs sensing or physical navigation.